

Sistemas Distribuídos — Semana 06

Peer-to-Peer e Estudo de Caso Comparativo · Aulas 11 (08/09) e 12 (10/09) · Prof. Leandro Borges

O que é uma arquitetura P2P

Diferente do modelo cliente/servidor estudado nas semanas anteriores, em uma arquitetura peer-to-peer (P2P) não existe um servidor central fixo — cada nó, chamado de peer, pode atuar como cliente e servidor ao mesmo tempo, dependendo do momento.

Essa arquitetura traz vantagens importantes. Não há ponto único de falha: se um nó sai da rede, os demais continuam funcionando normalmente, já que não existe um servidor central cuja queda derrubaria todo o sistema. Há também escalabilidade natural: cada novo peer que entra na rede contribui com recursos próprios — banda, armazenamento, processamento — em vez de apenas consumir os de um servidor central. O desafio, por outro lado, é a coordenação: sem um coordenador central, localizar recursos específicos e manter consistência entre todos os nós se torna uma tarefa mais complexa.

P2P estruturado vs. não estruturado

No modelo não estruturado, os peers se conectam de forma essencialmente aleatória entre si. Buscar um recurso nessa rede exige inundá-la com pedidos (uma técnica chamada flooding) — é simples de implementar, mas pouco eficiente à medida que a rede cresce, já que o volume de mensagens de busca cresce junto. O Gnutella é um exemplo histórico dessa abordagem.

No modelo estruturado, os peers se organizam segundo uma topologia bem definida, tipicamente usando uma tabela de hash distribuída (DHT, Distributed Hash Table). Localizar um recurso se torna muito mais eficiente nesse modelo, ao custo de uma manutenção mais complexa da estrutura da rede quando peers entram e saem. Chord e Kademlia (este último usado pelo protocolo BitTorrent) são exemplos conhecidos de redes estruturadas.

Aplicações reais de P2P

O BitTorrent distribui pedaços de um mesmo arquivo entre milhares de peers simultaneamente, acelerando consideravelmente o download em comparação a baixar de um único servidor. Redes de blockchain, como Bitcoin e Ethereum, fazem cada nó manter uma cópia completa do razão (ledger) distribuído, validando transações de forma descentralizada, sem depender de uma autoridade central. O IPFS (InterPlanetary File System) é um sistema de arquivos distribuído que armazena e localiza conteúdo por hash do próprio conteúdo, em vez de depender de um servidor central que hospede o arquivo.

Estudo de caso: Sockets vs. RPC vs. RMI

Para fechar o panorama de comunicação estudado neste bimestre, vale comparar diretamente as três abordagens vistas: sockets, RPC e RMI. Em nível de abstração, sockets oferecem o nível mais baixo (o programador lida diretamente com bytes na rede), RPC oferece um nível médio (chamadas de função abstraem a rede) e RMI o nível mais alto (objetos remotos abstraem tudo). Em controle sobre o protocolo, é o oposto: sockets dão controle total sobre como os dados trafegam, RPC oferece controle parcial, e RMI, controle baixo — a maior parte fica escondida pela infraestrutura de objetos distribuídos. Em facilidade de uso, a ordem se inverte de novo:

sockets exigem mais código e cuidado do desenvolvedor, RPC é mais fácil de usar, e RMI, mais fácil ainda para quem já trabalha no paradigma orientado a objetos.

Não existe uma abordagem "melhor" de forma absoluta — a escolha depende de quanto controle o problema exige e de quanto tempo de desenvolvimento está disponível.

Referências

COULOURIS, G. et al. Sistemas Distribuídos: Conceitos e Projeto. 5. ed. Porto Alegre: Bookman.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.